

Terminal 3 ADK — Onboarding Findings

Written while completing the Quickstart and the Walkthrough on 12 August 2026, then extended while running a second contract end to end against the live testnet.

Every finding below has evidence you can check yourself: a file and line in the published SDK package, the vendored WIT in the official reference repo, or verbatim command output including the `request_id` the docs ask for.

Environment used throughout:

Tenant DID	<code>did:t3n:f21dce7928980eeea7dc93618b91f602a80fe1c4</code>
Node	<code>https://cn-api.sg.testnet.t3n.terminal3.io</code>
SDK	<code>@terminal3/t3n-sdk@4.35.1</code>
Rust	1.89.0, target <code>wasm32-wasip2</code>
wasm-tools	1.255.0
Node.js	24.13.0
OS	Windows 11 (10.0.26200)

Fifteen findings. Five stop a new developer outright, five cause real confusion, four waste time, and one is a proposal rather than a defect.

Findings T-11 through T-15 came from actually running a contract on testnet, and they are the ones I would read first — none of them are discoverable by reading the documentation. T-15 is the one that ended the exercise: the credit grant ran dry.

Blockers

T-01 — The first code sample in the Quickstart cannot run

`T3nClientConfig.trustAnchor` is a required field. The Quickstart sample omits it, so the constructor throws `T3nConfigError` before any network traffic happens. A developer following the page hits this on their first attempt with no hint that the sample itself is wrong.

From `node_modules/@terminal3/t3n-sdk/dist/index.d.ts`:

```
interface T3nClientConfig {
  wasmComponent: WasmComponent;
  baseUrl?: string;
  trustAnchor: TrustAnchorOrUnsafe; // no `?` — required
  handlers?: Partial<Handlers>;
}
```

The claim page at go.terminal3.io/adk-community carries the same defect in its own sample, so the problem exists on two separate surfaces.

What works. Fetch the anchor first. `fetchTrustedManifest` is documented as never returning an unverified anchor, so making it the default path in the docs costs nothing:

```
setEnvironment("testnet");
const nodeUrl = getNodeUrl();
const trustAnchor = await fetchTrustedManifest("testnet");
```

Verified against testnet: HTTP 200, valid operator signature, 3 peers, 1 RTMR3 measurement, signed `2026-08-11T14:14:45Z`.

Suggested fix. Add `trustAnchor` to the sample on both the Quickstart and the claim page.

T-02 — The handler set in the Quickstart is incomplete for the handshake

The sample passes `EthSign` alone. The SDK exports `createDefaultHandlers`, whose own doc comment describes it as the handler set the handshake requires — it supplies the ML-KEM key handler and the randomness source in addition to signing.

What works:

```
handlers: {
  ...createDefaultHandlers(nodeUrl, trustAnchor),
  EthSign: metamask_sign(ethAddress, undefined, apiKey),
}
```

Suggested fix. Show the spread form. It is one extra line and it removes a class of failure that is hard to attribute.

T-03 — The docs contradict the WIT and the official reference repo

The common-errors page states that `tenant.did()` is already a string. It is not. From `wit/deps/host-tenant-1.0.0/package.wit`:

```
tenant-did: func() -> list<u8>;
```

Twenty raw bytes. The official `Terminal-3/z-tenant-flight` repo encodes it with `hex::encode` before using it to build a map name. The Walkthrough sample follows the docs instead, so it does not compile.

This one is worse than a typo, because map names are derived from the DID. Get the encoding wrong and every later KV call addresses a map that does not exist.

Suggested fix. Correct the common-errors page and show `hex::encode(&tid)` in the Walkthrough.

T-12 — Importing `signing@2.1.0` silently kills the contract

This cost the most time of anything in this exercise, and nothing in the documentation could have prevented it.

`host:interfaces/signing@2.1.0` is declared in `host-interfaces-2.1.0/package.wit` at line 159, alongside `logging`, `kv-store`, and the HTTP interfaces. Nothing marks it as unavailable to tenant contracts.

A contract whose world imports it registers successfully and reports a normal contract id. Then **every** call to it fails:

```
RPC Error: Internal error [754f19f6-b496-4f2d-b475-add668c7d85e]
```

Additional request ids for the same condition: `e1c1193d-1638-4885-b122-cb2233adc3bc`.

Three details make this hard to diagnose:

1. Registration succeeds. The rejection happens at instantiation, later.
2. `contracts.logs` returns `{"entries": [], "next_seq": null}` — the contract

never starts, so it cannot log anything about its own failure.

1. Functions that never touch signing fail identically. `list-receipts` only scans a KV map, and it returned the same opaque 500.

So the signal points nowhere near the cause. I bisected by removing interfaces one at a time. Dropping `signing` — changing nothing else — turned the opaque 500 into a precise, useful contract error:

```
access denied: TenantContract(did:t3n:f21dce.../617) cannot read map "z:f21dce...:receipts"
```

The reference repo does not import `signing` either, which in hindsight was the clue. But absence in one example is not documentation.

Suggested fix. Two things, either of which would have saved hours:

- Reject the *registration* when the world imports an interface the tenant

runtime does not provide, and name the interface in the error.

- Mark availability per interface in `host-interfaces`. If `signing` is

cluster-internal, say so in its doc comment.

T-11 — `tenant.claim()` returns HTTP 500 on an already-provisioned tenant

The SSO claim page already creates the tenant row and credits the account. Calling `tenant.claim()` afterwards does not report that state — it fails:

```
RPC Error: Internal error [28113ed7-3a9d-45ef-98b8-b664fd9eb421]
RPC Error: Internal error [d8dbb69a-758f-4a4a-a67e-1e52b2f4b742]
```

Two attempts, two request ids, same result. Meanwhile the tenant is demonstrably healthy — every read succeeds:

```
tenant.me()      -> { "status": "active", "label": "testnet-dev", ... }
getBalance()     -> { "available": 19989922328, "reserved": 0 }
contracts.list() -> []
```

The docs' own guidance for a generic 500 is to retry once and then report it. I retried; same outcome.

This matters more than it looks, because the shape of the response suggests the call is meant to be idempotent — an `already-admitted` status with null `granted_credits`. A caller who trusts that and puts `claim()` at the top of a setup script is blocked on a step that was already finished.

Workaround now in the repo: read `tenant.me()` first and only claim when `status !== "active"`. See `scripts/src/02-claim-tenant.ts`.

Suggested fix. Return `already-admitted` instead of 500.

Confusing

T-13 — Re-registering a contract invalidates every map ACL, silently

Map ACLs bind to the numeric contract id, not to the contract name. Every registration mints a **new** id. So the moment you re-register — which you must do for any code change — all maps that allowed the old id start refusing the new one.

Observed across ten registrations of the same contract during this exercise:

Version	Contract id
0.1.0	615
0.1.1	617
0.1.2	618
0.1.3	619
0.1.4	620
0.1.5	621
0.1.6	638
0.1.7	639
0.1.8	641
0.1.9	653

Note that the ids are neither contiguous nor predictable — 621 to 638 skips sixteen, and 641 to 653 skips twelve. So there is no way to precompute the next one and pre-authorise it.

Ten registrations is also what exhausted the credit grant, which is finding T-15. The two compound: iterating on a contract forces re-registration, re-registration forces an ACL fix-up, and the registrations themselves drain the allowance.

Maps created against 615 rejected 617 with:

```
access denied: TenantContract(did:t3n:f21dce.../617) cannot read map "z:f21dce...:receipts"
```

Credit where due: **this error message is excellent**. It names the principal, the id, the map, and the operation. It is the clearest error I saw all day. The problem is purely that nothing warns you the relationship exists, so on a first encounter it reads as though the map itself is broken.

`maps.update(tail, { readers, writers })` fixes it, and it is not mentioned in any documentation page.

Suggested fix. State in the Walkthrough that contract ids are per registration and that map ACLs must be re-pointed after every re-register. A `maps.update` snippet next to `maps.create` would close it.

Related: `MapCreateInput.readers` carries a genuinely good warning in its doc comment — omitting it creates a map nobody can read, with no error. That warning deserves to be on the docs page, not only in the type definition.

T-14 — `kv-store.scan` returns raw CAS pointers; `kv-store.get` resolves them

The WIT promises key/value pairs:

```
scan: func(
  map-name: string,
  start: list<u8>,
  end: list<u8>,
  limit: u32,
) -> result<list<tuple<list<u8>, list<u8>>>, string>;
```

For values large enough to be offloaded to content-addressed storage, the value you get back from `scan` is not the value. It is a pointer envelope: the ASCII magic `T3VR` followed by JSON.

Captured from inside the contract, first 64 bytes, hex and lossy UTF-8:

```
543356527b2276616c75655f636964223a5b32362c32372c31393332c...
T3VR{"value_cid": [26, 27, 193, 186, 180, 45, 202, 168, 81, 195, 32, 170, 0, 2
```

`kv-store.get` on the exact same key resolves the pointer and returns the original bytes. I proved the asymmetry with two functions of the same contract reading the same row in the same deployment:

- `verify-receipt` reads via `get`, parses the JSON, recomputes the digest →

```
{"valid": true}
```

- `list-receipts` reads via `scan`, parses the same way → `expected value at line

1 column 1`, with the value length reported as 311 bytes rather than zero

The failure mode is nasty because the host raises no error. `scan` succeeds. The bytes are the wrong bytes.

Workaround now in the repo: detect the `T3VR` prefix and re-read that key with `get`. See `contract/z-tenant-nullstamp/src/list.rs`.

Suggested fix. Either resolve pointers in `scan` the way `get` does, or state in the WIT comment that scanned values may be pointers and must be resolved with a follow-up `get`. The current signature promises something it does not always deliver.

T-15 — The credit grant is spent far faster than the claim page implies

The claim page advertises the 20,000 test credit grant as "enough for 25 agents and ~5,000 protected actions". In practice this exercise exhausted it completely with ten contract registrations and roughly a dozen contract calls.

The account ended at zero, and the node then refuses further work:

```
InsufficientCredit (account=f21dce7928980eaea7dc93618b91f602a80fe1c4,
                    required=10000000000, available=0)
[824ac0e3-0a2d-4d4a-b1d4-d1733f9e28a4]
```

Note the shape of that number. A single contract execution asks to lock 10,000,000,000 against a total grant of 20,000,000,000, so on the face of it the entire allowance covers two concurrent calls. Balances read from `getBalance()` along the way:

Point in the exercise	available
Start	19,989,922,328
After the first receipt	19,939,779,025
After v0.1.5	8,078,980,220

After v0.1.7	3,502,747,402
After v0.1.9	0

The largest single drop coincides with a run of re-registrations, which is worth connecting to finding T-13: because contract ids are minted per registration and map ACLs bind to the id, iterating on a contract *forces* repeated registration. So the two findings compound — the workflow the platform requires is the workflow that drains the grant.

Nothing warns you as the balance falls, and the failure arrives as a hard stop mid-pipeline rather than as a warning at 20% remaining.

Suggested fix. Three things, in order of value:

- State the real cost of a registration and of a call on the claim page, or express

the grant in the same units as the quotas so the two can be compared.

- Warn on `getBalance()` or in the register response when the remaining balance is

below a few calls' worth.

- Reconsider the per-call lock. Locking half the total grant for one execution makes

the advertised "~5,000 protected actions" unreachable by construction.

Worth saying: this one cost me the tail of the exercise. The final pipeline run completed steps 01 through 10 and died on step 11, which is only an export helper. The receipt itself was already exported and remains verifiable — offline verification needs no credits at all, which is the one silver lining and, in fairness, a decent advertisement for the design.

T-04 — The tenant claim step is not documented anywhere

`tenant.claim()` exists in the SDK as the self-serve path on testnet, including the test-credit grant. Neither the Quickstart nor the Walkthrough mentions it. A developer who never calls it may or may not have a tenant row, depending on how their DID was created — and the failure surfaces much later, at `maps.create`.

See also T-11, which is what happens when you do call it.

T-05 — The documented order cannot be followed to completion

The `maps.create` example uses `contractId` in its reader and writer sets without saying where that number comes from. The only source is the result of `contracts.register`, which the Walkthrough sequences *after* map creation.

Following the page in order leaves you holding a placeholder for a value you cannot have yet.

Suggested fix. Register first, then create maps. Or state that maps can be created with a placeholder ACL and re-pointed afterwards — which, per T-13, is the operation everyone will need anyway.

T-06 — Installing the SDK reports a critical vulnerability

`npm install @terminal3/t3n-sdk` in an empty project:

The critical one is a Zip Slip in `decompress`, reached through `jco → componentize-js → weval`. A fix is available upstream. Not blocking for testnet work, but it is the first thing a security reviewer at an adopting company will see.

Time sinks

T-07 — Useful SDK surface is absent from the docs

Not mentioned on any page, all present and working:

Method	Why it matters
<code>contracts.logs</code>	The first tool you want when a call fails opaquely
<code>maps.entrySet</code>	Seeds a KV entry directly, far nicer than hand-rolling <code>executeControl</code>
<code>maps.entryGet</code>	Reads it back; <code>null</code> is the deliberate "absent" signal
<code>maps.getStatus</code>	Confirms a map is <code>active</code> before depending on it
<code>maps.update</code>	The fix for T-13

`contracts.logs` is the sharpest omission. It is what turned T-12 from guesswork into a bisection — the empty log was the evidence that the contract never started.

T-08 — The error-handling example swallows the two variants that matter most

The `format_http_error` sample ends in a catch-all arm, which collapses `PlaceholderDenied` and `PlaceholderNoUserContext` into a bare `error`. The WIT comments explain that these two are deliberately distinguished: one means the marker is not permitted, the other means there is no user session to resolve it against. Those call for completely different fixes.

Nullstamp handles each separately in `src/issue.rs`.

T-09 — Broken links and an empty page

Two URLs linked from the docs index return 404. The payroll-agent example page resolves but contains no explanation — only links out.

T-10 — Version numbers in the reference repo disagree

`world.wit` says `0.4.0` while `Cargo.toml` and `CONTRACT_VERSION` say `0.4.1`, and a test name still refers to the older version. Confusing at exactly the wrong spot, since the version must increase on every re-registration.

Proposal

U-01 — `set-claims-digest` deserves a documentation page of its own

Its WIT comment says the digest is planted in the transaction's Merkle leaf "so clients can verify receipts offline."

That sentence answers the hardest question about confidential computing: how can anyone outside check work performed somewhere they cannot see? No ADK page discusses it.

Nullstamp is built on this capability, and the result holds up. A receipt issued inside the enclave on testnet, exported and checked by a separate TypeScript program that imports no SDK, opens no session, and makes no network request:

```
receipt_id      : rcpt_8fb056c99bbea7e655d72075
digest recorded : 8fb056c99bbea7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3
digest computed : 8fb056c99bbea7e655d72075e37e428c8fbaade3c79b32fb737a6c4bef82e4d3

RESULT: valid. Digest recomputed outside the node and it matches.
```

Give this capability a page with a worked canonical-form example, and the pitch for T3N changes from "trust the enclave" to "compute it yourself and compare." That is a materially stronger claim, and the machinery for it already ships.

Reproducing all of this

```
git clone https://github.com/bryankwandou/nullstamp
cd nullstamp/contract/z-tenant-nullstamp
cargo test --target "$($rustc -vV | sed -n 's/^host: //p')"
cargo build --target wasm32-wasip2 --release
wasm-tools component wit target/wasm32-wasip2/release/z_tenant_nullstamp.wasm

cd ../../scripts
npm install
npm run preflight                # no developer key needed

cp .env.example .env             # add T3N_API_KEY
npm run step:01                  # through step:11
npm run verify:offline -- ../submission/live-receipt.json
```

Reported by Bryan Kwandou — nayrbryangaming01@gmail.com